

COMPX304

Advanced Networking & Cyber Security

Lab Exercise 9: Cache hits and misses and their timing

CPU Caches are memory devices used by the CPU to store items that are frequently used. Since CPU caches are closer to the CPU and faster than main memory they help reduce the amount of time needed to fetch the items. Over the years manufacturers have developed a hierarchy of CPU caches. The first exercise in this lab is to find out how many caches the CPU on your machine has and their respective sizes.

1. Run the `lscpu` command on the terminal of a linux machine and find out the number of caches and their sizes.

The amount of time a CPU takes to fetch data can be measured in terms of CPU cycles and indicates the time/cycles the CPU spent being unproductive. Obviously, reducing the CPU cycles used for fetching items would result in the CPU becoming more efficient. In the next exercise we would run a program that will show the amount of CPU cycles needed to fetch items from the main memory vs the CPU cache.

2. Compile the `cachetime.c` program as follows

```
gcc -march=native cachetime.c
```
3. Execute the program. With the help of the comments in the program describe the results.
4. Based on the results choose a suitable THRESHOLD number, so that you can say if CPU cycles required to access an element are below the THRESHOLD then the element is cached otherwise it is in the main memory.

In the next exercise we will perform a technique called FLUSH + RELOAD to retrieve a secret value from the CPU cache. This is analogous to the light bulb example and our objective is to get the secret value. The technique works as follows

- a. FLUSH: We first prepare an array of 256 elements. Before accessing the secret we flush the entire array out of the cache memory to make sure none of the array elements are cached.
- b. Let's say the secret that we want to store in the cache is a number S . We now access the array element at position S . This will result in the S -th array element being cached in the CPU cache.
- c. RELOAD: Assume that our memory is now erased and we do not remember the value S . To recover the secret, we will access all the 256 array elements and check which of

the array elements is possibly cached. The array element that appears to be cached is our secret number.

The program `flush.c` has the code for the FLUSH+RELOAD technique. Keep in mind that this technique is not 100% accurate. To increase our confidence in the results we get, we can perform multiple runs of the same experiment and see cumulative results.

5. Compile the `flush.c` program as follows and execute it

```
gcc -march=native flush.c
```
6. The secret variable in `flush.c` holds the secret. You can choose it to be any number between 1 and 255.
7. What do you observe in the output of `flush.c` in terms of CPU cycles. How many CPU cycles does retrieving the S -th item take, where S is your secret value in `flush.c`? Repeat this experiment multiple times. On average is the time taken to retrieve the S -th item less, more or the same as every other item in the array?

--- End of Lab 9 ---